

SQL in Python

Python supplies the values and decides what happens next.

Gregory S. DeLozier, PhD

Running the Example

```
python3 db-example.py --db practice.db
```

The script rebuilds the pet table on every run.

Choosing a Database

```
import argparse
import sqlite3

ap = argparse.ArgumentParser()
ap.add_argument("--db", default="pets.db")
args = ap.parse_args()
connection = sqlite3.connect(args.db)
```

`args.db` supplies the filename.

Python Calls, SQL Statements

```
connection.execute("drop table if exists pet")  
connection.commit()
```

Python calls the method. SQLite interprets the string.

Inspecting the Tables

```
cursor = connection.execute("""
    SELECT name FROM sqlite_master
    WHERE type = 'table' ORDER BY name
""")
list_of_tables = [item[0] for item in cursor.fetchall()]
```

```
['pet', 'sqlite_sequence']
```

SQL and Values Travel Separately

```
connection.execute(  
    "INSERT INTO pet (name, kind, age, food) "  
    "VALUES (?, ?, ?, ?)",  
    ("Dorothy", "dog", 11, "peanut butter"),  
)
```

Four placeholders. Four values, in the same order.

Committing Changes

```
connection.execute(  
    "INSERT INTO pet (name, kind, age, food) "  
    "VALUES (?, ?, ?, ?)",  
    ("Whiskers", "hamster", 1, "hamster chow"),  
)  
connection.commit()
```

Dorothy and Whiskers precede this commit.

Fetching One Row

```
cursor = connection.execute("SELECT * FROM pet")  
row = cursor.fetchone()  
pprint(row)
```

```
(1, 'Dorothy', 'dog', 11, 'peanut butter')
```

A stored record becomes a Python tuple.

Fetching the Remaining Rows

```
cursor = connection.execute(  
    "SELECT name FROM pet ORDER BY id"  
)  
first = cursor.fetchone()  
rest = cursor.fetchall()
```

```
first: ('Dorothy',)  
rest: [('Whiskers',)]
```

The One-Element Tuple

```
cursor = connection.execute(  
    "SELECT sql FROM sqlite_master "  
    "WHERE type='table' AND name=?",  
    ("pet",),  
)  
row = cursor.fetchone()  
pprint(row[0] if row else "")
```

("pet",) is a tuple. ("pet") is a string.

An Error Does Not Fix Itself

```
try with a wrong name  
Caught sqlite error: no such table: petz  
Caught sqlite error: no such table: petz
```

Catching the exception lets the program continue.

Repeating the same misspelling fails again.

Deleting Stash

```
connection.execute(  
    "DELETE FROM pet WHERE name=?",  
    ("Stash",),  
)  
connection.commit()
```

The condition can match more than one record.

Updating Dorothy

```
connection.execute(  
    "UPDATE pet SET age=?, food=? WHERE name=?",  
    (12, "pretzels", "Dorothy"),  
)  
connection.commit()
```

SET supplies changes. WHERE selects records.

The Final Stored State

id	name	kind	age	food
1	Dorothy	dog	12	pretzels
2	Whiskers	hamster	1	hamster chow
3	Sandy	cat	9	tuna
4	Maxwell	cat	11	tuna

A new process can retrieve these records.

Finishing with the Connection

```
connection = sqlite3.connect("practice.db")
try:
    cursor = connection.execute(
        "SELECT name FROM pet ORDER BY name"
    )
    pprint(cursor.fetchall())
finally:
    connection.close()
```

Closing is not a substitute for committing.